

CRUD de alumnado por capas



En este proyecto vamos a trabajar en el aula un CRUD con:

- JDK 17
- Maven
- JUnit 5
- persistencia en CSV de inicio
- arquitectura por capas
- herencia y clase abstracta
- validaciones con expresiones regulares
- terminología asociada al proyecto y en la que trabajamos

1. Qué vamos a construir

Vamos a desarrollar una aplicación de consola para gestionar alumnado.

La aplicación permitirá:

- crear alumnos
- listar alumnos
- buscar alumnos por identificador
- modificar alumnos
- eliminar alumnos

Los datos se guardarán en un fichero CSV.

2. Qué conceptos vamos a trabajar

CRUD

Permite practicar las operaciones básicas sobre datos:

- **Create:** crear
- **Read:** leer
- **Update:** actualizar
- **Delete:** borrar

Programación orientada a objetos

Usaremos clases, objetos, herencia y abstracción.



Arquitectura por capas

Separaremos el programa en partes con responsabilidades distintas:

- **app:** interacción con el usuario
- **servicio:** lógica de negocio
- **persistencia:** acceso a datos
- **ficheros:** gestión física del fichero
- **validacion:** reglas de validación



Persistencia en fichero CSV

Trabajaremos cómo convertir objetos Java en líneas de texto y viceversa.

Validación con expresiones regulares

Comprobaremos que los datos tengan el formato correcto antes de guardarlos.

3. Estructura del proyecto

```
crud-alumnado-csv/  
├── pom.xml  
├── README.md  
├── src/  
│   ├── main/  
│   │   ├── java/  
│   │   │   └── com/docente/  
│   │   │       ├── app/  
│   │   │       ├── ficheros/  
│   │   │       ├── modelo/  
│   │   │       ├── persistencia/  
│   │   │       ├── servicio/  
│   │   │       └── validacion/  
│   │   └── resources/  
│   └── test/  
│       └── java/  
│           └── com/docente/
```

4. Guía paso a paso de lo que vamos a construir

Paso 1. Definir el dominio

Primero pensamos qué representa nuestro programa.

Queremos gestionar alumnado, así que necesitamos modelar:

- una **Persona**
- un **Alumno**

Qué concepto se trabaja

- abstracción
- herencia
- reutilización

Para qué se utiliza

Para representar mejor la realidad del problema.

Beneficio

Evita repetir atributos comunes y mejora el diseño.

Paso 2. Crear **Persona** como clase abstracta

Persona será la clase base.

Contendrá:

- **identificador**
- **nombre**
- **edad**

Y tendrá un método abstracto **toCSV()**.

Qué concepto se trabaja

- clase abstracta
- métodos abstractos

Para qué se utiliza

Para definir una estructura común que otras clases van a heredar.

Beneficio

Centraliza lo común y obliga a las subclases a completar lo específico.

Paso 3. Crear **Alumno** como subclase de **Persona**

Alumno heredará de **Persona** y añadirá el atributo **curso**.

Qué concepto se trabaja

- herencia
- uso de **super**
- especialización

Para qué se utiliza

Para crear un tipo concreto de persona con características propias.

Beneficio

Permite extender el modelo sin duplicar código.

Paso 4. Crear la capa **validacion**

Añadiremos todas aquellas funciones que consideremos necesarias, con una expresión regular por ejemplo.

Ejemplo de formato válido:

```
ALU001
ALU123
```

Qué concepto se trabaja

- validación de datos
- expresiones regulares
- responsabilidad única

Para qué se utiliza

Para impedir que se guarden datos incorrectos.

Beneficio

Aumenta la calidad del programa y evita errores posteriores.

Paso 5. Crear la capa **persistencia** a través de una interfaz

Primero se define el contrato:

- **IAumnoRepositorio**

Después se crea la implementación concreta:

- **AlumnoRepositorioCSV**

Qué concepto se trabaja

- interfaces
- abstracción
- desacoplamiento

Para qué se utiliza

Para que la capa de servicio dependa de un contrato y no de una clase concreta.

Beneficio

Permite cambiar la implementación sin romper el resto del código.

Paso 6. Crear la clase abstracta **FicheroAbstract**

Esta clase se encargará de:

- localizar la carpeta del fichero
- crearla si no existe

- crear el fichero si no existe
- dejar preparado el recurso físico

Qué concepto se trabaja

- **File**
- gestión de recursos
- reutilización técnica

Para qué se utiliza

Para no repetir el mismo código en cada clase de persistencia.

Beneficio

Centraliza la parte técnica del acceso al fichero.

Paso 7. Implementar la persistencia CSV

Primero definimos la interfaz:

- **ISlumnoRepositorio** y después la implementación para CSV **AlumnoRepositorioCSV** se encargará de:
- leer líneas del CSV
- saltar la cabecera "identificador,nombre,edad,curso";
- convertir cada línea en un **Alumno**
- guardar la lista de alumnos en el fichero

Qué concepto se trabaja

- lectura de ficheros
- escritura de ficheros
- transformación texto ↔ objeto

Para qué se utiliza

Para conservar la información aunque el programa se cierre.

Beneficio

El alumnado entiende cómo persistir datos sin usar aún base de datos.

Paso 8. Crear la capa **servicio**

Primero definimos la interfaz:

- **IAlumnoServicio**

Después la implementación:

- `AlumnoServicioImpl`

El servicio combinará:

- validación
- reglas de negocio
- acceso a persistencia

Qué concepto se trabaja

- lógica de negocio
- coordinación entre capas

Para qué se utiliza

Para decidir si una operación puede realizarse.

Beneficio

Evita que la lógica quede mezclada con el menú o con el fichero.

Paso 9. Crear la capa `app`

En `Main` construiremos el menú por consola.

Desde esta capa se llamará al servicio para realizar las operaciones.

Qué concepto se trabaja

- interacción por consola
- uso de `Scanner`
- integración del sistema

Para qué se utiliza

Para ofrecer una interfaz sencilla al usuario.

Beneficio

Mantiene el programa organizado: la app pregunta y muestra, pero no decide la lógica ni gestiona el CSV.

Paso 10. Añadir pruebas con JUnit 5

Crearemos pruebas para comprobar:

- que el validador funciona
- que el servicio impide duplicados
- que el repositorio CSV crea y guarda correctamente

Qué concepto se trabaja

- testing automatizado
- comprobación de comportamiento

Para qué se utiliza

Para verificar que el código funciona y seguir refactorizando con seguridad.

Beneficio

Reduce errores y mejora la confianza en el proyecto.



11. Explicación de los paquetes

`com.docente.modelo`

Representa los datos del dominio.

- `Persona`
- `Alumno`

`com.docente.validacion`

Centraliza las reglas de validación.

- `ValidadorPersona`
- `ValidadorAlumno`

`com.docente.ficheros`

Gestiona la parte física del fichero.

- `FicheroAbstracta`

`com.docente.persistencia`

Accede a los datos almacenados.

- `IAlumnoRepositorio`
- `AlumnoRepositorioCSV`

`com.docente.servicio`

Contiene la lógica de negocio.

- `IAlumnoServicio`
- `AlumnoServicioImpl`

`com.docente.app`

Contiene el punto de entrada del programa.

- `Main`

6. Cómo ejecutar el proyecto

Compilar

```
mvn clean compile
```

Ejecutar tests

```
mvn clean test
```

Ejecutar la aplicación

```
mvn exec:java -Dexec.mainClass="com.docente.app.Main"
```

También se puede ejecutar `Main` directamente desde el IDE.

###. Qué ventajas tiene este diseño

- el código está separado por responsabilidades
 - se entiende mejor
 - se mantiene mejor
 - permite ampliar el proyecto con menos cambios
 - facilita las pruebas
 - introduce al alumnado en un estilo de desarrollo más profesional
-